

實習生月報

PDO 資料路徑重建

CoE / SDO 交易重建

PDO Mapping / Assignment 關聯整理

SyncManager / FMMU 資料路徑分析

報告期間

2026.08

MONTHLY REPORT

本月工作大綱



08/xx - 08/xx

理解封包與 EtherCAT 100%

- 使用 Wireshark 觀察從上電抓的原始 EtherCAT 封包
- 整理 Broadcast、AP、FP 等指令對 Slave 的操作方式
- 整理 Master 對 Slave ESC 暫存器的讀寫流程
- 理解 CoE、SDO、PDO 與 Object Dictionary 的關係
- 建立本月主線：CoE / SDO → PDO Mapping / Assignment → SyncManager → FMMU

08/xx - 08/xx

建立 Slave 拓樸關係 80%

- 理解 Master 如何透過 Auto Increment / Configured Address 指定 Slave
- 理解 Configured Station Address 與 Configured Station Alias 的差異
- 設計 AP Datagram 配對、Slave Discovery 模型與分析流程

設計 Analyzer 驗證 Agent 20%

- 建立 Stage 3 Result Check
- 由 Capture evidence 驗證 Slave Count、Topology、Address 與 Identity

08/xx - 08/xx

Mailbox / CoE / SDO 70%

- 在 Datagram Record 中加入 CoE / SDO decoded fields
- 重建 SDO Initiate Download Request / Response Transaction
- 依 SDO Index / SubIndex 辨識 PDO Mapping / Assignment 操作
- 重建 RxPDO / TxPDO Mapping Configuration
- 重建 0x1C12 / 0x1C13 PDO Assignment Configuration

完善 Agent 驗證流程 30%

- 整理 TShark EtherCAT / CoE / SDO 查詢與 JSON 結構規格
- 將常用 Capture 查詢封裝成 Python tools，提供 LLM 多種驗證方式

08/xx - 08/xx

SyncManager / FMMU 分析 100%

- 在 Datagram Record 中加入 SyncManager / FMMU decoded fields
- 重建 SyncManager Configuration
- 重建 FMMU Configuration
- 驗證 PDO Assignment 與 SM2 / SM3 的設定關係
- 驗證 FMMU Physical Address 與 SyncManager RAM 區段的對應

SECTION / 01

01 上電流程與 ESC Register 解析

追蹤啟動封包，建立 ESC 暫存器存取與解析脈絡

上電封包操作流程

01 Broadcast Initialization

BRD / BWR / BRW
All Slaves

02 Auto Increment Addressing

APRD / APWR / APRW
Topology Position

03 Configured Station Address

APWR / ADO 0x0010
Fixed Address

04 Fixed Position ESC Access

FPRD / FPWR / FPRW
Configured Address + ADO

Broadcast
Initialization

BRD / BWR / BRW

Auto Increment
Addressing

APRD / APWR / APRW

Configured Station
Address

APWR / 0x0010

Fixed Position ESC
Access

FPRD / FPWR / FPRW

上電後，Master 先透過 Broadcast Command 對所有 Slave 執行共通操作，再利用 Auto Increment Addressing 依鏈路位置逐站存取 Slave。完成 Configured Station Address 配置後，Master 即可使用 Fixed Position Command，透過固定站址指定 Slave，再利用 ADO 存取該 Slave 的 ESC Register。



SII EEPROM 讀取流程

SII EEPROM Read 操作步驟

- 01

檢查 EEPROM 狀態

讀取 ESC Register 0x0502~0x0503，確認 EEPROM Interface 不處於 Busy 狀態，並確認沒有 Error，才能繼續 EEPROM 存取。
ESC Register · 0x0502~0x0503 · EEPROM Control / Status Register
- 02

指定 SII Word Address

將欲讀取的 SII EEPROM Word Address 寫入 ESC EEPROM Address Register 0x0504~0x0507。
0x0008 → Vendor ID 起始 Word；0x000A → Product Code 起始 Word
ESC Register · 0x0504~0x0507 · EEPROM Address Register
- 03

送出 EEPROM Read Command

透過 EEPROM Control / Status Register 的 Command 欄位送出 Read Command，要求 ESC 開始讀取前一步指定的 SII EEPROM Word。
0x0502[10:8] · Read Command = 001
- 04

等待 EEPROM Read 完成

再次檢查 EEPROM Control / Status Register，等待 Busy 清除，並確認沒有 EEPROM Error。
0x0502 · Busy / Error Status
- 05

讀取 EEPROM Data

EEPROM Read 完成後，對 ESC EEPROM Data Register 讀取資料。Master 可使用 APRD，ADO = 0x0508，取得先前指定 SII Word 對應的 EEPROM Data。
ESC Register · 0x0508~0x050F · EEPROM Data Register

實際 Capture 中，Master 的 APWR 可由 ADO 0x0502 開始，一個 payload 同時涵蓋 EEPROM Control / Status 與 0x0504~0x0507 EEPROM Address；之後再以 APRD 讀取 ADO 0x0508 的 EEPROM Data。因此這 5 個步驟是操作邏輯，不代表每一步一定對應獨立的一個 EtherCAT Datagram。

ESC Register 與 SII EEPROM Word Address 對照表

ESC Register Address	How Master accesses the EEPROM interface	
SII EEPROM Word Address	Where Slave information is stored	
Address Space	Address	Meaning
ESC Register	0x0502~0x0503	EEPROM Control / Status
ESC Register	0x0504~0x0507	EEPROM Address
ESC Register	0x0508~0x050F	EEPROM Data
SII EEPROM Word	0x0008~0x0009	Vendor ID
SII EEPROM Word	0x000A~0x000B	Product Code

0x0502 / 0x0504 / 0x0508 是 ESC Register Address；
0x0008 / 0x000A 是 SII EEPROM Word Address。

Auto Increment → Slave Discovery

[STEP 01]

[STEP TITLE]

[STEP BODY]

[STEP 02]

[STEP TITLE]

[STEP BODY]

[STEP 03]

[STEP TITLE]

[STEP BODY]

[STEP 04]

[STEP TITLE]

[STEP BODY]

[STEP 05]

[STEP TITLE]

[STEP BODY]

[RESULT TITLE]

[RESULT ITEM]

[RESULT ITEM]

[RESULT ITEM]

[RESULT ITEM]

[RESULT ITEM]



分析問題與目標

01 PROBLEM

封包解析提供的是獨立的 **CoE / SDO Record**

單一 Datagram 無法直接表示一次完整的 SDO object access，
Request 與 Response 必須跨封包建立關聯。

- Request / Response 分散
- Index / SubIndex 資訊分散
- Result / Abort 狀態需要整合

02 GOAL

重建完整的 **SDO Transaction**

將 Mailbox / CoE / SDO 欄位重新組合，
建立可供後續 PDO Mapping 分析使用的 transaction-level model。

- Request ↔ Response
- Index / SubIndex
- Access Direction
- Result / Abort

INPUT

Mailbox / CoE Datagram

PROCESS

Transaction Reconstruction

OUTPUT

SDO Transaction Model



SDO Transaction Reconstruction Flow



EtherCAT Spec 重點整理

01 KEY CONCEPT	Core concept or mechanism Summarize the core mechanism, state transition, or protocol rule that defines the behavior described in the specification. Note what the device or analyzer should observe when this rule is applied.
02 IMPORTANT FIELDS	Registers / objects / parameters Record register addresses, object indices, data types, access rules, and parameter constraints. Highlight which fields are read-only, writable, or dependent on operating state.
03 DATA FLOW	How data moves through the system Trace how the request, datagram, mapped memory, and application data move through the system. Mark the boundaries where timing, ownership, or synchronization affects the result.
04 ENGINEERING NOTE	Implementation or analyzer relevance Connect the specification detail to source code, packet analysis, validation steps, or observable system behavior. Record the implementation risk or evidence that should be checked.



EtherCAT 圖解分析

FIGURE / 04



[Diagram Canvas]

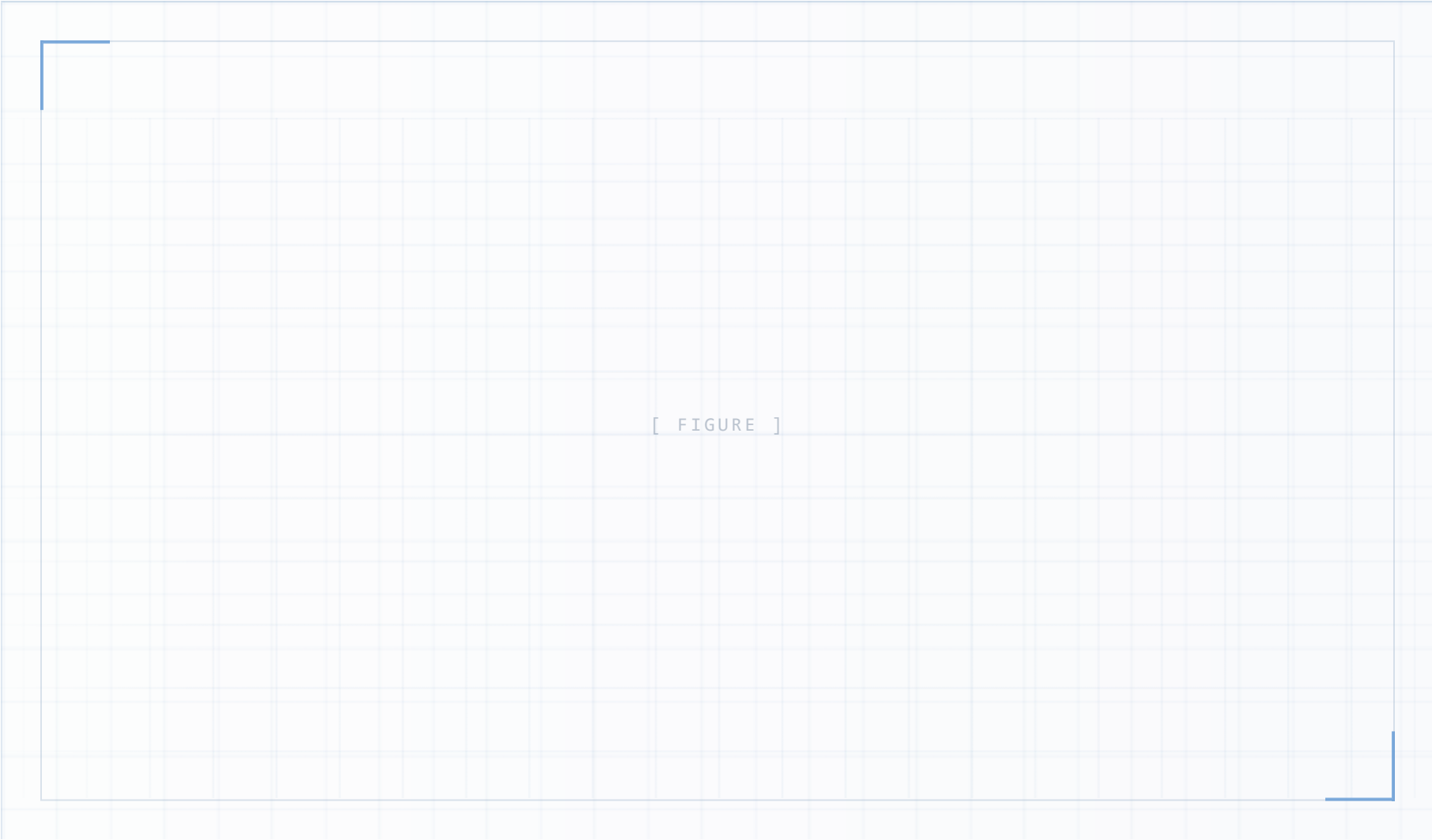
Class Diagram / Architecture Diagram / Data Path Figure

REPLACE WITH IMG / SVG / FIGURE / HTML DIAGRAM

FOCUS POINTS

- 01 Diagram Purpose**
Explain the main relationship between modules.
- 02 Key Components**
Highlight the major classes or architecture blocks.
- 03 Reading Order**
Describe how to read the diagram efficiently.
- 04 Implementation Relevance**
Connect the diagram to the current development work.

EtherCAT 資料路徑概覽



KNOWLEDGE POINTS

01

FMMU

Logical Address
→ ESC local

02

SyncManager

Consistent data
exchange control

03

Process RAM

Stores mapped
process data

04

DATA PATH

Logical Address → FMMU →
SM → RAM → Application

